

Actuate — Master Implementation Specification

Buildable Reference: Architecture, Tech Stack, Interfaces, and Sequenced Build Plan

Consolidates v1 (L0–L8 pipeline), v2 (VLA + dexterous), v3 (10-paper merge), v3.1 (EgoScale + EgoVerse). Implementation-ready. Spec only — nothing herein is built; every benchmark-before-lock and GPU-validation point is flagged.

0. How to read this document

This is the document you build from. For each layer it gives: **status** (built / extend / net-new), **tech stack** (specific libraries and models, with "benchmark-before-lock" marked), **public interface** (the library API and CLI surface), **inputs/outputs** (typed against the canonical schema in §3), and **verification gate** (what must pass, and what needs real GPU/real-data validation before it counts as done).

Build order is §8. If you want to start today, start at **Phase 0 (§8)** — the canonical schema in §3 is the freeze point everything else compiles against.

Two non-negotiables carried from the whole project:

1. **CLI/API-first.** Everything is a Python library function first. The Typer CLI and the FastAPI service are thin consumers of that library — never dependencies of it. If a capability only works through the CLI or only through the service, it is wrong.
 2. **Verify against real data.** A component is "done" only when tested against real data, and every correctness test must be confirmed to fail against a broken version, not merely pass against the current one. Each layer below states its verification gate. "Looks right" is not a status.
-

1. Build status ledger (what exists vs. what's new)

Component	v1 status	Target state	Delta type
Ingestion, sync, QA	Built	Add 6th rig (DexUMI exoskeleton), aligned-capture mode, SLAM/IMU ego-motion, phone tier	Extend

Hand pose (MediaPipe)	Built	Swap to WiLoR → MANO (HaMeR fallback)	Extend (model swap)
Object detect/track (Grounding DINO + SAM2)	Built	Keep; add FoundationPose 6-DoF object pose	Extend
Depth (Depth-Anything-V2)	Built	Swap to rig-specific: UniDepthV2 (mono) / FoundationStereo (stereo); benchmark vs MoGe-2+FLOW3R+GeoCalib	Extend (model swap)
Primitives / phases / episode segmentation	Built	Keep; phase boundaries double as subgoal-image anchors	Keep
VLM language grounding	Built	Add multi-paraphrase, structured hand+object+action captions, LLM-as-judge gate	Extend
EIS certification	Built	Map to π 0.7 episode metadata; add strategy-alignment item	Extend
PII redaction + fail-closed consent gate	Built	Untouched — hard-blocking	Keep
Dataset packaging	Built (format-generic)	LeRobot v3 + RLDS exporters , rich norm stats, Stage-I/Stage-II tiers	Net-new export
FastAPI service + web dashboard	Built	Surface new fields/formats	Extend (consumer)
Sensor-fusion arbiter (L2)	—	Trust-weighted arbiter + per-finger contact confidence	Net-new
Canonical schema (MANO + retargeted joints, both action spaces, metadata)	Partial	Frozen, versioned	Net-new / rework
Retargeting engine (L5)	—	Arm (VN+flow), finger (GeoRT), reconciliation, sim validation	Net-new

2. Package architecture (CLI/API-first)

2.1 Repository layout

actuate/ # importable library — the source of truth

```

__init__.py
config/          # Pydantic-settings, YAML profiles, rig registry
schema/          # canonical data contracts (§3) — msgspec/pydantic
io/              # MCAP, PyAV, Zarr/HDF5, Parquet readers/writers
ingest/          # L0
perception/      # L1: hand/ object/ depth/ slam/ pose submodules
fusion/          # L2
canonical/       # L3
certify/         # L4: quality/ pii/ consent/ llm_judge/ strategy_align
retarget/        # L5: arm/ finger/ reconcile/ sim_validate/ embodiments
language/        # L6
package/         # L7: lerobot/ rlds/ normalize/ tiers
feedback/        # L8
cli/             # Typer app (thin)
service/         # FastAPI app (thin consumer)
tests/
  unit/          # fast, mock-free where possible
  integration/   # real-data fixtures, small real clips
  fixtures/      # tiny real capture samples per rig
pyproject.toml   # uv/pip; extras per layer to keep deps optional

```

Dependency direction: `cli/` and `service/` import from the layer packages; layer packages import only from `schema/`, `io/`, `config/`, and each other in pipeline order. Nothing in a layer imports from `cli/` or `service/`. Enforce with an import-linter contract in CI.

Optional heavy deps: perception/retarget models (torch, CUDA libs, sim engines) live behind extras (`actuate[perception]`, `actuate[retarget]`, `actuate[sim]`) so the core library and exporters install without a GPU stack.

2.2 CLI surface (Typer)

Each layer is one command group; every command is a thin wrapper over a library call.

```

actuate ingest run      --rig <rig> --in <path> --out <store>
actuate perceive hands  --in <store> --model wilor
actuate perceive depth  --in <store> --model auto      # auto = rig-specific
actuate perceive objects --in <store> --prompts prompts.yaml
actuate perceive slam   --in <store>                  # moving-camera rigs
actuate fuse run        --in <store>
actuate canonical build --in <store> --out <canonical> # freezes to schema vN
actuate certify run     --in <canonical>              # quality + consent + judge + strategy
actuate retarget arm    --in <canonical> --embodiment <name>
actuate retarget finger --in <canonical> --embodiment <name>
actuate retarget validate --in <canonical> --embodiment <name> # sim replay
actuate language annotate --in <canonical>
actuate package lerobot --in <canonical> --out <dataset> --tier stage1|stage2

```

```
actuate package rlds    --in <canonical> --out <dataset>
actuate run all        --config profile.yaml          # full pipeline
```

Design rules: every command reads/writes the canonical store (§3), is idempotent, records provenance, and takes `--dry-run` and `--limit N` (process N episodes) for cheap real-data spot-checks. Long/GPU stages print an explicit "requires GPU" banner and a resumable checkpoint path.

2.3 Config

Pydantic-settings with layered YAML profiles (`base.yaml` → `rig/<rig>.yaml` → `embodiment/<name>.yaml`). A **rig registry** maps each capture rig to its native channels, ego-motion method, depth model, and trust ordering; an **embodiment registry** maps each robot to its kinematic model (URDF), hand type/DoF, control modes, and camera config. These two registries are the extension points for new rigs and new customer robots.

3. Canonical schema (the freeze point — build this first)

The canonical representation is the contract every downstream layer compiles against. Freeze and version it (`schema_version`) before building L4–L7. Use `msgspec.Struct` (fast, typed, Parquet/Zarr-friendly) or Pydantic v2.

Per-frame record:

Field	Type	Source	Required by
<code>t</code>	float64 (s, canonical clock)	L0	all
<code>rig, episode_id, frame_idx</code>	str/str/int	L0	all
<code>images.<cam></code>	ref to chunked MP4 (consistent cam names)	L0	VLA output
<code>camera_pose</code>	SE(3), world frame	L1 SLAM / MPS	egocentric reframing
<code>hand.<L/R>.mano</code>	MANO params (β fixed, θ 15-PCA)	L1 WiLoR	EgoVLA-style pretrain (intermediate)
<code>hand.<L/R>.keypoint_s_3d</code>	21×3, camera frame	L1	fingertip retarget input

<code>hand.<L/R>.wrist_pose</code>	SE(3), camera frame	L1	wrist/EEF action
<code>finger_joints_human</code>	per-finger joint angles	L2 (glove) / L1 (vision)	dexterous
<code>finger_joints_robot space</code>	joint angles in robot hand space	L2 (DexUMI exoskeleton)	dexterous (measured GT)
<code>object.<id>.pose</code>	SE(3)	L1 FoundationPose	interaction, contact
<code>object.<id>.mask</code>	RLE/ref	L1 SAM2	grounding
<code>depth.<cam></code>	ref to depth map + uncertainty	L1	metric 3D
<code>contact.<finger></code>	confidence $\in [0,1]$ + source enum	L2	dexterous prerequisite
<code>interaction_state</code>	enum {STATIC, GRASPED_L/R/BOTH, MOVING}	L2	fusion
<code>confidence.<field> + provenance.<field></code>	float / source-tag	all	certification

Per-episode / per-segment record:

Field	Type	Source	Required by
<code>task</code>	str (imperative) + <code>task_paraphrases[]</code>	L6	VLA
<code>subtasks[]</code>	list of (span, instruction)	L6	π 0.7 subtask / subgoal anchor
<code>subgoal_frames[]</code>	frame refs at phase boundaries	L6	π 0.7 subgoal images
<code>action.human</code>	relative-SE(3) wrist + retargeted joints on canonical reference hand	L5	Stage-I pretrain target
<code>action.robot.<embodiment></code>	joint traj and EE traj, per control mode	L5	Stage-II / finetune

<code>control_mode</code>	enum {joint, ee} tag	L5	π 0.x contract
<code>finger_action_repr</code>	{absolute, relative} both available	L3	DexUMI robustness
<code>episode_meta.quality</code>	1–5 (from L4 EIS)	L4	π 0.7 metadata
<code>episode_meta.speed</code>	length in steps (binned)	L4	π 0.7 metadata
<code>episode_meta.mistakes[]</code>	per-segment flags	L4	π 0.7 metadata
<code>strategy_alignment.<embedding></code>	ok / flagged + reason	L4/L5	EgoVerse Robot-B guard
<code>norm_stats</code>	per-dim 1/99 percentiles (+ mean/std)	L7	training
<code>diversity.scene_id,</code> <code>diversity.demonstrator_id</code>	ids	L0	L7 diversity reporting
<code>tier</code>	{stage1_volume, stage2_anchor}	L7	delivery tiering
<code>effective_hours</code>	float (post-filter)	L7	scaling-law value
<code>consent, pii_status</code>	enum (fail-closed)	L4	hard gate
<code>schema_version</code>	str	—	migration

Key decisions baked in (from v3/v3.1): MANO is the *intermediate*; the delivered pretraining action target is **relative-SE(3) wrist + retargeted joints on a canonical high-DoF reference hand** (pick one, ~20+ DoF). Both control modes and both finger-action representations are carried. Episode metadata mirrors π 0.7. Strategy-alignment and consent are certification-gated.

Verification gate: round-trip a real processed episode through `canonical build` → Parquet/Zarr → reload, asserting bit-exact field recovery; and confirm the schema migration test fails when a required field is dropped.

4. Per-layer implementation

L0 — Ingestion & Sync · **actuate/ingest**

- **Status:** built; extend.
- **Tech stack:** MCAP (container), PyAV (decode), Polars + NumPy (tabular/sync), Zarr/HDF5 (frame store), Pydantic (rig manifests). Ego-motion: **ORB-SLAM3** (stereo+IMU rigs) or Aria MPS (Aria data). Phone tier: commodity iPhone, cloud VI-tracking.
- **New:** DexUMI-exoskeleton rig (robot-space encoder joints + tactile); **aligned-capture mode** (human rig shares camera intrinsics/extrinsics with a target robot — required for Stage-II anchor); multi-paraphrase slot reserved.
- **Interface:** `ingest.run(rig, src, store) -> IngestReport`; per-rig adapter classes implementing `RigAdapter` (declares native channels + ego-motion method).
- **Verification gate:** on a real multi-rig sample, assert every stream lands on one canonical clock within tolerance; **regression-test the known fps bug** (fixture where source fps $\neq$ metadata fps must be caught, not silently trusted).

L1 — Perception & Metric Reconstruction · **actuate/perception**

- **Status:** built (MediaPipe hands, Depth-Anything-V2); extend with model swaps.
- **Tech stack:**
 - Hand $\rightarrow$ MANO: **WiLoR** (primary), **HaMeR** (fallback). *Benchmark-before-lock vs current MediaPipe on real clips. GPU.*
 - Object detect/segment/track: **Grounding DINO + SAM2** (built, keep). **GPU.**
 - Object 6-DoF pose: **FoundationPose** (new). **GPU.**
 - Depth (rig-specific): **UniDepthV2** (mono, metric + intrinsics + uncertainty) / **FoundationStereo** (stereo, mm-level) / **MoGe-2 + FLOW3R + GeoCalib** (EgoInfinity stack). *Benchmark-before-lock — do not assume the impl-spec default; EgoInfinity does not use UniDepthV2. GPU.*
 - Ego-motion: ORB-SLAM3 / Aria MPS (see L0).
- **Interface:** `perception.hands.run(store, model="wilor")`, `.depth.run(store, model="auto")`, `.objects.run(store, prompts)`, `.pose.run(store)`, `.slam.run(store)`. Each writes typed fields + per-field confidence + provenance.
- **Verification gate:** each model validated on real frames against held-out ground truth or cross-sensor agreement (e.g., stereo depth vs checkerboard, per EgoLive's mm-level protocol). **All L1 model swaps need real GPU validation before they count as done.**

L2 — Sensor-Fused Interaction Refinement · **actuate/fusion**

- **Status:** net-new.
- **Tech stack:** typed state machine (Python enums + transition table), SciPy/NumPy signal ops. Trust-weighted arbiter with ordering `measured_robotspace (DexUMI) > measured_human (glove) > gripper aperture > vision-primary > vision-fallback`.

- **Output:** `interaction_state`, `contact.<finger>` confidence — the hard prerequisite for the dexterous branch.
- **Interface:** `fusion.run(store) -> FusionReport`.
- **Verification gate:** on real glove + vision episodes, assert measured channels override vision when present; unit test the arbiter with a broken-priority variant that **must** fail.

L3 — Canonical Representation · `actuate/canonical`

- **Status:** partial; **rework to the frozen §3 schema**.
- **Tech stack:** msgspec/Pydantic v2 structs; Parquet (episode/metadata) + Zarr (dense per-frame); camera-centered stable-frame reprojection (NumPy).
- **Interface:** `canonical.build(store, out, schema_version) -> CanonicalDataset`.
- **Verification gate:** §3 round-trip test; stable-frame reprojection validated against a moving-camera clip (future actions must land in the current device frame consistently).

L4 — Certification · `actuate/certify`

- **Status:** built (EIS, PII, consent); extend.
- **Tech stack:** existing EIS scorer → emit $\pi 0.7$ `quality/speed/mistakes`; **PII/consent fail-closed gate (untouched)**; **LLM-as-judge** caption-consistency gate (hand/object/action/global — EgoLive protocol) via a VLM; **strategy-alignment** check fed from L5 (flag if retargeting forces an undemonstrated strategy — EgoVerse Robot-B guard).
- **Interface:** `certify.run(canonical) -> CertificationReport` (per-episode pass/flag + metadata written back).
- **Verification gate:** the consent gate must be re-tested against the **auto-consent workaround** class of bug — a fixture that attempts to bypass consent must be blocked; strategy-alignment flag must trigger on a known mismatched episode.

L5 — Cross-Embodiment Retargeting · `actuate/retarget`

- **Status:** net-new.
- **Tech stack:**
 - **Arm/wrist branch:** SE(3)-equivariant **Vector-Neuron** root-frame estimator with **flow-matching** objective, trained in **MuJoCo** (procedural OU→IK→spline data; ~1.5–2 hr/robot on a single 3060 per EgoInfinity). IK: **cuRobo** (GPU) or **Pinocchio/PyRoki** (CPU). Candidate clustering (k-means over SE(3)) + scoring (convergence, manipulability, joint-limit margin, smoothness). **GPU (training)**.
 - **Finger branch (kinematic):** **GeoRT** (public code) — per-finger fingertip→joint MLP, unsupervised, ~3–5 min/hand on a 3060; needs per-human ~5-min C-space calibration + per-hand robot C-space sample. Contact-blind → feeds reconciliation. **GPU (light)**.

- **Canonical reference-hand retarget:** standing retarget of MANO→canonical high-DoF hand (EgoScale-style) as a delivered Stage-I output.
- **DexUMI bypass:** exoskeleton rigs skip finger retargeting (already robot-space).
- **Contact-consistency reconciliation:** arm-IK wrist + finger posture must agree at contact.
- **Sim validation:** MuJoCo (default) / **Isaac Lab** (scale) replay with collision, joint-limit, contact-stability, **no-slip** checks. DexMachina's contact-guided idea informs *scoring* here — **not** wholesale DexMachina RL (that is a flagged research track, not on this path).
- **Interface:** `retarget.arm.run(canonical, embodiment), .finger.run(...), .reconcile.run(...), .sim_validate.run(...)`; embodiment adapters in `retarget/embodiments/<name>` (URDF + hand model + control modes).
- **Verification gate:** replay retargeted trajectories in sim and assert no-slip/collision/joint-limit pass on a real episode; the reconciliation test must fail on a deliberately contact-inconsistent trajectory. **Retargeting model training needs real GPU validation.**

L6 — Language & Task Annotation · **actuate/language**

- **Status:** built; extend.
- **Tech stack:** grounding VLM (e.g., Qwen-VL-class) for structured **hand+object+action** captions; multi-paraphrase generation (1 human + N LLM paraphrases, per TRI LBM); phase boundaries (built) → subtask spans + subgoal-frame anchors ($\pi 0.7$); closed action ontology (built) beneath open task names. Consistency scored by the L4 LLM-as-judge gate.
- **Interface:** `language.annotate(canonical) -> AnnotationReport`.
- **Verification gate:** LLM-as-judge consistency on a labeled real subset; a hallucinated-caption fixture must be flagged, not passed.

L7 — Packaging & Delivery · **actuate/package**

- **Status:** built (format-generic); **net-new exporters.**
- **Tech stack:** **LeRobot v3** exporter (chunked Parquet + per-camera chunked MP4 + tying metadata) primary; **RLDS/TFDS** exporter secondary (Open-X convention — *verify field layout against the real library, not from memory*); normalization: default **1st/99th-percentile** → **[-1,1]** ($\pi 0.5$ /EgoVerse), ship raw percentiles (+ mean/std) so customers on 2/98-per-timestep (TRI LBM) or z-score (EgoMimic) can re-derive; **Stage-I volume tier** and **Stage-II aligned-anchor tier**; dual-space delivery (human + retargeted robot); optional masked/overlay variants (EgoMimic); scene- and demonstrator-diversity reported separately (EgoVerse); **effective_hours** recorded (EgoScale scaling-law value).
- **Interface:** `package.lerobot.export(canonical, out, tier), package.rlds.export(...), package.normalize.compute(canonical)`.

- **Verification gate:** load the exported LeRobot v3 dataset with LeRobot's own loader and run one real training step, including the normalization round-trip (TRI LBM: normalization dominates). A schema-valid-but-untrainable export must be caught by this gate — do not assert schema-correctness in place of a real load+train.

L8 — Feedback · actuate/feedback

- **Status:** built; keep with one reframe.
- **Tech stack:** dataset versioning + QC hooks (built). Reframe: honestly-graded failure/low-quality episodes route to delivery as π 0.7 metadata-labeled robustness data (not just quarantine); consent/PII failures stay hard-blocked.

Service & Dashboard · actuate/service

- **Status:** built; extend as a **consumer**. FastAPI endpoints surface retarget-eligibility, per-finger contact, episode metadata, export formats, tier toggle. The web dashboard consumes the API. Neither is a dependency of the library. (EgoDB is a useful reference for the query surface: SQL over tasks/embodiments/scenes/sources.)

5. Cross-cutting concerns

- **Provenance & confidence** on every derived field (already a project norm) — carried through L3, consumed by L4.
- **Idempotence & resumability:** every stage checkpoints and can resume; --limit N runs a cheap real-data slice.
- **Testing discipline:** tests/unit (fast, avoid vacuous mocks), tests/integration (tiny **real** clips per rig in tests/fixtures). CI import-linter enforces the CLI/API-first dependency direction. Every "proves correctness" test carries a companion broken-variant test that must fail.
- **GPU boundary:** perception models, retargeting training, and sim validation are GPU stages behind extras; the core library, schema, certification (non-VLM parts), and exporters run CPU-only so value ships without a GPU stack.

6. Compute & GPU-validation map

Stage	Compute	Needs real-GPU validation before "done"
L1 hands (WiLoR), objects (DINO+SAM2), depth, pose (FoundationPose)	GPU inference	Yes — validate each swap on real frames

L1 SLAM (ORB-SLAM3)	CPU/GPU	Yes — on real moving-camera clips
L2 fusion, L3 canonical, L4 (non-VLM), L7 exporters	CPU	No (unit + real-data integration)
L4 LLM-as-judge, L6 VLM captions	GPU/API	Yes — consistency on labeled real subset
L5 arm estimator training (VN+flow, MuJoCo)	GPU (single 3060 sufficient per EgoInfinity)	Yes
L5 finger (GeoRT)	GPU (light)	Yes
L5 sim validation (MuJoCo/Isaac)	GPU	Yes
L7 LeRobot round-trip train step	GPU (small)	Yes — the load+train gate

Free-tier (Kaggle/Lightning) covers L5 arm/finger training and most L1 validation; serverless rental for Isaac-scale sim validation and the LeRobot train-step gate if needed.

7. Third-party choices to lock via benchmark (not from memory)

1. **Depth model** — UniDepthV2 vs FoundationStereo vs MoGe-2+FLOW3R+GeoCalib, per rig, on real Actuate captures.
2. **Hand model** — WiLoR vs HaMeR vs incumbent MediaPipe, real clips.
3. **RLDS field layout & OpenVLA discrete-token option** — confirm against the real libraries (training-knowledge, not a fresh read).
4. **Canonical reference hand** — pick the ~20+ DoF hand used as the retarget target for the Stage-I human pretraining representation.
5. **IK backend** — cuRobo (GPU) vs Pinocchio/PyRoki (CPU) for L5.
6. **EgoScale scaling-law constants** — refit on Actuate's own certified data if attaching predicted-value estimates; do not reuse **0.024** / **0.003**.

8. Sequenced build plan

Each phase names its deliverable and its verification gate. Value ships before the hardest component is done.

Phase 0 — Schema & input analysis (days, CPU). Deliverable: frozen §3 canonical schema (`schema_version=1`), rig + embodiment registries, GeoRT-I/O and glove/DexUMI-output analysis folded in. Gate: §3 round-trip test passes; drop-a-required-field migration test fails as expected.

Phase 1 — Canonical build against the freeze (CPU). Deliverable: `canonical build` producing the frozen schema from already-processed v1 outputs; stable-frame reprojection. Gate: real episode round-trips; reprojection validated on a moving-camera clip.

Phase 2 — LeRobot v3 / RLDS exporter with $\pi 0.7$ rich context (mostly CPU; small GPU for the train-step gate). SHIPS VALUE. Deliverable: exporters + normalization (1/99 default, raw percentiles), episode metadata, multi-paraphrase, both control modes, Stage-I tier; training-ready gripper/arm data independent of dexterous work. Gate: **load with LeRobot's own loader + run one real training step + normalization round-trip.** This is the first shippable milestone.

Phase 3 — L1 model swaps + L2 fusion (GPU). Deliverable: WiLoR→MANO, rig-specific depth, FoundationPose, SLAM ego-motion; trust-weighted arbiter + per-finger contact confidence (glove/DexUMI first). Gate: each model validated on real frames; arbiter broken-priority test fails.

Phase 4a — Kinematic dexterous track (GPU). Highest near-term risk. Deliverable: arm branch (VN+flow, MuJoCo) + GeoRT finger branch + reconciliation + sim no-slip validation; DexUMI-exoskeleton data flows end-to-end first (bypasses retargeting, de-risks). Gate: sim replay passes no-slip/collision/joint-limit on a real episode; reconciliation fails on a contact-inconsistent trajectory.

Phase 4b — Functional/RL research track (GPU, parallel/after, flagged). Deliverable: DexMachina-style contact-guided refinement in sim on glove-rig data. Not gated in front of launch; privileged-state sim-to-real is an open risk.

Phase 5 — Language rich-context + aligned-anchor + delivery hardening. Deliverable: multi-paraphrase + subtask/subgoal anchors + LLM-as-judge gate; **aligned-capture mode (L0) + strategy-alignment cert (L4)** enabling the Stage-II anchor tier; dual-space delivery; scene/demonstrator diversity reporting. Gate: hallucinated-caption fixture flagged; strategy-mismatch fixture flagged; Stage-II anchor exports pass the Phase-2 load+train gate.

Certification & consent (Phase 0 onward, always-on): the fail-closed consent/Pll gate is wired from the first canonical build and re-tested against the auto-consent-bypass bug class at every phase.

9. Honest status

Nothing in this document is built or tested; it is the spec you build from. The v1 pipeline (through packaging) exists and is the foundation; everything marked net-new or rework in §1 is unbuilt. Every model swap, retargeting component, sim validation, and the LeRobot

train-step gate needs real GPU/real-data validation before it counts as done — flagged in §6. The benchmark-before-lock items in §7 must not be resolved from memory. As the build proceeds, each component's status will be reported honestly as tested-against-real-data vs. unit-tested vs. only-written, with correctness tests confirmed to fail against a broken version.